

SC EXAM PREP — PREDICTED 2026

Web/アプリケーション脆弱性

OWASP／SQLi・XSS・CSRF・SSRF／セキュアコーディング。午前IIの用語判別から午後の設計・実装対策まで、頻出6論点を5問で体系学習する。

Q1・SSRF

メタデータ窃取／宛先偽装

頻出 ★★★

Q2・SQLi

プレースホルダの本質

頻出 ★★★

Q3・XSS

格納型XSSの識別

頻出 ★★★

Q4・CSRF

SameSite属性の役割

頻出 ★★☆

Q5・Path

ディレクトリトラバーサル

頻出 ★★☆

 基礎概念 5枚  問題演習 5問  統合サマリー  2026 春・秋 想定

※ 過去問傾向(令和3～7年)と最新動向をもとにAIが作成した予想問題です。実際の試験問題ではありません。最終確認を推奨します。



学習マップ — 5問の相互関係

MAP

「注入(インジェクション)」と「セッション・認可」の2系統で切り分ける学習動線

Q1

SSRF

サーバに任意宛先へ要求させ、クラウドのメタデータ(169.254.169.254)から認証情報を窃取

許可リスト + IP検査



Q2

SQLi

入力をSQLの構文要素として解釈させる注入。プレースホルダで構文とデータを分離

バインド機構



Q3

XSS

出力に混入したスクリプトが被害者ブラウザで実行。格納型は永続化・無差別配信

出力エスケープ



Q4

CSRF

他サイト起点の状態変更要求。SameSiteでCookie自動送信を抑制、トークンで照合

トークン + SameSite



Q5

Traversal

「../」でパス外参照。識別子化 + 正規化後に基準ディレクトリ配下を検証

canonicalize

STEP 1 | WHY

OWASP Top10 の攻撃を「注入系」と「セッション・認可系」の全体像で理解する

STEP 2 | HOW

5問で「どの脆弱性か」を経路と結果から判別し、ひっかけ選択肢を切る

STEP 3 | WHAT

空-雨-傘で対策(構文分離・出力エスケープ・許可リスト・正規化検証)へ落とす

① 基礎概念 — Webアプリ脆弱性 分類マップ

CONCEPT

攻撃の「起点」と「本質(何が壊れるか)」の2軸で本デッキ5問を配置

系統 / 本質	注入系 (構文・命令の混入)	セッション・認可系 (なりすまし・越境)
利用者入力 経由	Q2 SQLインジェクション／Q3 XSS(反射・DOM・格納)／OSコマンドインジェクション	セッションフィクセーション／セッションハイジャック
サーバ挙動 経由	Q1 SSRF(サーバに任意宛先へ要求)／Q5 ディレクトリトラバーサル	オープンリダイレクト／認可不備(IDOR)
他サイト 経由	HTTPヘッダインジェクション	Q4 CSRF／クリックジャッキング

核心: 注入系は「データが構文・命令として解釈される」ことが本質で、対策は構文とデータの分離/エスケープ。セッション・認可系は「本人性・権限の検証不足」が本質で、対策はトークン照合・属性・正規化検証。この2系統に落とし込めば4択も午後も切れる。

② 基礎概念 — 混同しやすい概念 比較表

似て見えるが「起点・成立条件・対策」が異なる5ペア

CONCEPT

対象概念	● 概念A	● 概念B（混同パターン）
XSSの分類	格納型：スクリプトがサーバ側に永続化され閲覧者へ無差別配信	反射型/DOM型：その要求限りで応答反映(反射)、DOM操作過程で実行(DOM)
Cookie属性	HttpOnly：JSからの読取を禁止しXSSでのCookie窃取を緩和	SameSite：他サイト起点のCookie自動送信を抑制しCSRFを緩和(Secure=HTTPS限定送信)
SQLi対策の本質	プレースホルダ：入力をデータとして固定し構文構造を変えない	エスケープ/危険文字除去：表層的で漏れ・多重表現で回避され得る
SSRFと類似	SSRF：サーバに任意宛先へ要求させる(内部・メタデータ狙い)	オープンリダイレクト：利用者ブラウザを任意サイトへ誘導
パス系の攻撃	ディレクトリトラバーサル：正規化後に基準配下か検証	OSコマンドインジェクション：シェルを介さないAPI利用で根絶

③ 基礎概念 — 必須用語集

Webアプリ脆弱性5問を解く8用語

CONCEPT

SSRF Server-Side Request Forgery

サーバに任意宛先へHTTP要求を送らせる。クラウドのIMDS(169.254.169.254)から一時認証情報を窃取する経路に悪用。

 **OWASP Top10 A10:2021**

格納型XSS Stored / Persistent XSS

スクリプトがDB・ログ等に永続化され、閲覧者へ継続的・無差別に配信・実行される。反射型・DOM型と区別。

 **OWASP A03:2021 Injection**

SameSite属性

クロスサイト要求へのCookie自動送信を抑制(Lax/Strict)しCSRFを緩和。トークンの代替ではなく補強。

 **RFC 6265bis**

ディレクトリトラバーサル Path Traversal

「../」等でパス外を参照。ファイル名を識別子化、または正規化(canonicalize)後に基準配下を検証して根絶。

 **CWE-22**

プレースホルダ / バインド機構 Prepared Statement

SQLの構文構造を事前確定し、後続の入力値を常にデータとして扱う。テーブル名・列名はバインド不可で許可リスト対処。

 **SQLi の本質対策**

CSRF Cross-Site Request Forgery

被害者の認証済みセッションを悪用し、他サイト起点で意図しない状態変更要求を送らせる。トークン照合が本質対策。

 **OWASP A01:2021 関連**

HttpOnly / Secure属性

HttpOnly=JSからのCookie参照禁止(XSS窃取緩和)。Secure=HTTPS接続時のみ送信(盗聴緩和)。

 **RFC 6265**

IMDSv2 Instance Metadata Service v2

セッション指向・トークン必須(PUT)・ホップ制限でSSRF単独からの認証情報窃取を困難化。v1は無効化推奨。

 **クラウド基盤側の緩和**

④ 基礎概念 — 脆弱性 × 根本対策 マトリクス

「本質対策」と「表層策(不十分)」を取り違えない

CONCEPT

脆弱性(規格)	根本対策(正解語彙)	不十分・ひっかけ
SSRF (A10:2021)	宛先の許可リスト + 名前解決後IP検査(内部・リンクローカル遮断) + リダイレクト再検証	単なるURL文字列フィルタ。IMDSv2化・権限最小化は 基盤側の緩和 で併用。
SQLi (A03:2021)	プレースホルダ(バインド機構)で構文とデータを分離	危険文字のエスケープ/除去のみ、通信のTLS化(注入とは無関係)。
XSS (A03:2021)	出力時のコンテキスト依存エスケープ、装飾許可は許可リスト型サニタイズ + CSP	入力エスケープのみ、ブラックリスト方式の個別除去は回避されやすい。
CSRF (A01関連)	推測困難なCSRFトークンの発行・照合 + SameSite属性で補強	Referer/Origin検証のみは限定的。SameSite単独で完全代替は不可。
Path Traversal (CWE-22)	入力を識別子に限定、または正規化後に基準ディレクトリ配下を検証	「../」の単純除去。エンコード・多重表現で回避される。
セッション管理	ログイン直後の再発行、HttpOnly + Secure、タイムアウト	セッションIDをURLに付与(Referer・履歴・ログで漏えい)。

💡 頻出の軸: 「注入系は**構文/データ分離・出力エスケープ**」「セッション認可系は**トークン・属性・正規化検証**」。**表層策(危険文字除去・入力エスケープのみ・URL文字列フィルタ)**を正解に見せる4択のひっかけが定番。



試験のコツ & 頻出ひっかけ

Exam Strategy & Common Pitfalls

TIPS

✓ 解くコツ

1. XSSは**3分類で判別**：永続化=格納型／応答反映=反射型／DOM操作過程=DOM型。
2. SQLi対策の本質は「**入力をデータとして固定し構文構造を変えない**」=プレースホルダ。
3. Cookie属性は役割で暗記：HttpOnly=XSS窃取緩和／SameSite=CSRF緩和／Secure=盗聴緩和。
4. SSRFは「**サーバに要求させる**」+許可リスト+**名前解決後IP検査**が正解語彙。

⚠ よくあるひっかけ

- XSS対策を「**入力エスケープ**」で済ませたつもり→出力処理漏れで成立。
- SSRFとオープンリダイレクトの混同：SSRFは**サーバ**、リダイレクトは**ブラウザ**が動く。
- 「../」の**単純な文字列除去**は多重表現・エンコードで回避され不十分。
- SameSiteやReferer/Origin検証を**CSRFの本質対策**として単独で選ばせる。

合言葉：「起点(どこから)×本質(何が壊れるか)」で分類 → 対策は「**構文分離・出力エスケープ・許可リスト・正規化検証・トークン照合**」の5語彙に集約される。

クラウド環境のインスタンスメタデータエンドポイント(例:**169.254.169.254**)へアプリケーションサーバから意図しない**HTTP**リクエストを送信させ、一時的な認証情報を窃取する攻撃の名称はどれか。

ア クリックジャッキング

イ サーバサイドリクエストフォージェリ(**SSRF**)

ウ セッションフィクセーション

エ ディレクトリトラバーサル

正解 イ ア=**iframe**で操作誘導／ウ=**セッションID**固定化／エ=**パス外ファイル**参照

☁ 空 (事実)

サーバに任意宛先へ要求させ、リンクローカルのIMDS(169.254.169.254)から一時認証情報を取得できた。

🌧 雨 (解釈)

SSRFはメタデータサービスや内部システムへの到達経路として悪用される。窃取した認証情報でIAM権限を悪用される。

☂ 傘 (対策)

宛先の許可リスト化、**名前解決後IP**の検査で内部・リンクローカル宛てを遮断、IMDSv2への移行。

プレースホルダ(バインド機構)を用いたSQL文の組み立てが、SQLインジェクション対策として有効である最も本質的な理由はどれか。

ア SQL文が実行前に暗号化され、通信経路上で解読されないため

イ 入力値がSQL文の構文要素ではなくデータ(値)としてのみ扱われ、構文構造が変化しないため

ウ 危険な文字列を検出した時点でDBMSがトランザクションをロールバックするため

エ アプリケーションからDBへの接続がTLSで保護され改ざんが検知されるため

正解 イ ア・エ=通信保護でSQLi無関係/ウ=そのような自動ロールバック機構はない

空 (事実)

SQL文の構文構造を事前にコンパイル確定させ、後からバインドする入力値は常にデータとして扱う。

雨 (解釈)

入力の引用符やSQL断片が構文として解釈されず注入が成立しない。アプリのエスケープ処理に依存しない点が本質。

傘 (対策)

プレースホルダを徹底。テーブル名・列名など構文要素の動的指定はバインド不可のため許可リストで別途対処。

📁 実務メモ: 「エスケープ」ではなく **構文とデータの分離**が正解語彙。動的な識別子(列名・ソート順)を受け取る箇所だけは許可リストで固める、まで書ければ午後で加点。

OWASP Top 10(2021)で「Injection」に分類されるXSSのうち、格納型(蓄積型)XSSの説明として適切なものはどれか。

ア 攻撃者が仕込んだスクリプトがDBやログなどサーバ側に永続化され、当該データを閲覧した利用者のブラウザで実行される

イ URLパラメタに含まれるスクリプトが応答にそのまま反映され、被害者がリンクを踏んだ際に実行される

ウ サーバへのスクリプト送信を伴わず、クライアント側JavaScriptが入力をDOMへ反映する過程で実行される

エ 正規サイトを透過的なiframeで覆い、利用者の操作を誘導してクリックさせる

正解 ア イ=反射型XSS／ウ=DOM Based XSS／エ=クリックジャッキング

☁ 空 (事実)

スクリプトがサーバ側に永続化され、閲覧者へ継続的・無差別に配信される。

🌧 雨 (解釈)

格納型は被害が広範かつ持続的。反射型(その要求限り)・DOM型(クライアント内)と成立経路が異なる。

☂ 傘 (対策)

出力時のコンテキスト依存エスケープ。装飾許可なら許可リスト型サニタイズ+CSP+Cookieに HttpOnly。

Cookieに設定する属性のうち、**CSRF**の緩和を主目的として、他サイトを起点とするクロスサイトリクエストに**Cookie**を送信させないように制御する属性はどれか。

ア Secure属性

イ HttpOnly属性

ウ SameSite属性

エ Domain属性

正解 ウ ア=HTTPS接続時限定送信／イ=JSからの読取防止(XSS窃取緩和)／エ=送信対象ドメイン指定

☁ 空 (事実)

SameSite(Lax/Strict)はクロスサイト要求へのCookie自動送信を抑制する。

🌧 雨 (解釈)

CSRFの緩和に寄与するが緩和策であり、CSRFトークンによる照合を代替するものではない。

☂ 傘 (対策)

本質対策は推測困難な**CSRF**トークンの発行・照合。
SameSite属性を付与して補強する。

「../」などを含む入力で公開ディレクトリ外のファイルを不正に参照される脆弱性の名称と、その根本対策として適切な組合せはどれか。

ア OSコマンドインジェクション — シェルを介さないAPIの利用

イ ディレクトリトラバーサル — 入力を識別子に限定し、正規化後に許可ディレクトリ配下か検証する

ウ HTTPヘッダインジェクション — 改行コードの除去

エ オープンリダイレクト — リダイレクト先の許可リスト化

正解 イ ア・ウ・エ=別脆弱性と対策の正しい組だが本設問(バス外参照)には非該当

🔵 空 (事実)

バスの相対参照(../)を悪用し、基準ディレクトリ外のファイルを参照される。

🌧️ 雨 (解釈)

「../」の単純な文字列除去はエンコードや多重表現で回避され得るため不十分。

☂️ 傘 (対策)

ファイル名を直接受けず**識別子経由**で解決、または**正規化 (canonicalize)**後に基準配下であることを検証。



午後 記述式 — シナリオ要点

SaaS経費精算のSSRF／掲示板の格納型XSS・CSRF

記述

◆ 問1（春想定）連携先URL取得機能のSSRF

- 利用者入力URLへサーバが要求→**SSRF**。IMDS(169.254.169.254)で一時認証情報が画面表示。
- 被害 = 窃取した認証情報を外部から再利用し、IAM権限でオブジェクトストレージの他利用者データを読み取・改ざん・削除(横方向アクセス)。
- 入力対策 = ①宛先FQDNの許可リスト限定 ②名前解決後IP検査でループバック/リンクローカル/プライベート遮断 + リダイレクト再検証。
- 基盤側 = **IMDSv2化**・**v1無効化** + IAMロール権限最小化。

◆ 問2（秋想定）掲示板の格納型XSS・CSRF

- 自己紹介文にHTML保存許可→**格納型XSS**。**onerror**付imgが描画時に発火し**document.cookie**を外部送信。
- HttpOnly未設定でCookie窃取→セッション乗っ取り→なりすまし投稿。
- 装飾許可の対策 = 許可リスト型**HTMLサニタイズ**(実績あるライブラリ)を出力時に実施。
- CSRF根本対策 = セッション単位の推測困難な**トークン照合** + SameSiteで補強。

採点の分水嶺: ①SSRFと明記し被害を**認証情報の外部再利用による横断アクセス**まで具体化 ②対策は「危険文字除去」でなく**許可リスト + 名前解決後IP検証** ③XSSは**格納型 + 出力時サニタイズ** + HttpOnly ④CSRFは**トークン照合**まで踏み込めているか。



統合サマリー — 頻出ヒートマップ & 総括

出題可能性 × 難易度でWebアプリ脆弱性論点を俯瞰

SUMMARY

論点	出題可能性	難易度	対策語彙
SSRF / メタデータ窃取	最有力	高	許可リスト + IP検査
SQLi / プレースホルダ	最有力	中	構文/データ分離
XSS(格納/反射/DOM)	最有力	中	出力エスケープ
CSRF / Cookie属性	高	中	トークン + SameSite
Path Traversal / セッション管理	高	低	正規化検証

総括: Webアプリ脆弱性は**午前IIの用語・分類判別**(XSS3分類・Cookie属性の役割・SSRF/リダイレクトの主体差)と、**午後の設計・実装対策**(SSRF・格納型XSS・CSRF)の両面で狙われる。近年台頭の**SSRF(クラウド認証情報窃取)**と、依然定番の**XSS/SQLi**を最優先で固め、対策は必ず**表層策**でなく**本質対策**の語彙で書けるようにすること。

※本デッキはAIが過去問傾向・最新動向をもとに作成した予想問題です。正解・解説・図解は最終確認を推奨します。